WEEK #11 & #12

1# A DevOps engineer is managing a server configuration object and needs to verify how Python dictionaries handle access, modification, and deletion. Assume the variable server_config is initialized as: {"timeout": 300, "status": "active"}. Write a Python script that acts as a "Debug Console" to perform the following operations in order: • Read & Inspect: 1. Print the value associated with the key "status". 2. Safely try to get the value for the key "admin_email", defaulting to "Not Set" if it doesn't exist. 3. Print the total number of settings (len) in the dictionary. 4. Print a list of all Keys and a list of all Values. • Modify: 1. Replace the value of "timeout" with its negative (e.g., -300). 2. Add a new key/value pair: "max_connections": 100. • Clean Up: Remove the "timeout" key safely from the dictionary. • Sort: Print the remaining keys of the dictionary in alphabetical order.

```
# Take initial input
timeout = int(input("Enter timeout value: "))
status = input("Enter server status: ")

server_config = {"timeout": timeout, "status": status}

# --- Read & Inspect ---
print("Status:", server_config["status"])
print("Admin Email:", server_config.get("admin_email", "Not Set"))
print("Total settings:", len(server_config))
print("Keys:", list(server_config.keys()))
print("Values:", list(server_config.values()))

# --- Modify ---
server_config["timeout"] = -server_config["timeout"]
server_config["max_connections"] = int(input("Enter max connections: "))

print("\nAfter modification:", server_config)

# --- Clean Up ---
server_config.pop("timeout", None)
```

```
print("After cleanup:", server_config)


# --- Sort ---

print("Sorted Keys:", sorted(server_config.keys()))
```

```
Enter timeout value: 60
Enter server status: active
Status: active
Admin Email: Not Set
Total settings: 2
Keys: ['timeout', 'status']
Values: [60, 'active']
Enter max connections: 500

After modification: {'timeout': -60, 'status': 'active', 'max_connections': 500}
After cleanup: {'status': 'active', 'max_connections': 500}
Sorted Keys: ['max_connections', 'status']
```

2# A linguistic research tool needs to analyze the character distribution of different languages. Write a Python program to read a text string and create a Histogram (dictionary) that counts how often each letter appears. Example 1: • Input: "mississippi" • Output: {'m':1,'i':4,'s':4,'p':2} Example 2: • Input: "hello world" • Output: {'h':1,'e':1,'l':3,'o':2,' ':1,'w':1,'r':1,'d':1}

```
text = input("Enter a string: ")

histogram = {}

for char in text:

    histogram[char] = histogram.get(char, 0) + 1

print("Histogram:", histogram)
```

```
Enter a string: joao neves
Histogram: {'j': 1, 'o': 2, 'a': 1, ' ': 1, 'n': 1, 'e': 2, 'v': 1, 's': 1}
```

3# A corporate security system stores employee data as {Name: Badge_ID}. However, the security guard often finds a lost badge and needs to identify the owner using only the ID number. Write a Python program to perform a Reverse Lookup: • Input: A specific Badge ID (Value). • Action: Search through the dictionary to find and return the corresponding Name (Key). • Constraint: Handle the case where the ID does not exist. Example: • Data: {'Alice': 5001, 'Bob': 5002, 'Charlie': 5003} • Input: 5002 -> Output: "Bob" • Input: 9999 -> Output: "ID not found"

```
data = {}

n = int(input("Enter number of employees: "))


for i in range(n):
```

```
    name = input("Enter employee name: ")
    badge_id = int(input("Enter badge ID: "))
    data[name] = badge_id

search_id = int(input("Enter Badge ID to search: "))
found = False
for name, id in data.items():
    if id == search_id:
        print("Owner:", name)
        found = True
        break
if not found:
    print("ID not found")
```

```
Enter number of employees: 2
Enter employee name: amit
Enter badge ID: 3
Enter employee name: shah
Enter badge ID: 2
Enter Badge ID to search: 2
Owner: shah
```

4# An e-commerce pricing engine needs to apply a "Flash Sale". Create a dictionary Catalog_Prices containing the standard price of all items. You receive a second dictionary Sale_Updates that contains new prices only for items on sale. Write a Python script to merge Sale_Updates into Catalog_Prices. • Crucial Requirement: If an item exists in both, the value from Sale_Updates must overwrite the value in Catalog_Prices. Key Point: Merging updates existing data. The new value wins during a collision. Example: • Input: o Catalog = {'TV': 50000, 'Mouse': 500} o Sale = {'TV': 45000} • Output: {'TV': 45000, 'Mouse': 500} (TV price is updated, Mouse remains same)

```
Catalog = {}
Sale = {}
n = int(input("Enter number of catalog items: "))
for i in range(n):
    item = input("Enter item name: ")
    price = int(input("Enter price: "))
    Catalog[item] = price
```

```
m = int(input("Enter number of sale items: "))
for i in range(m):
    item = input("Enter sale item name: ")
    price = int(input("Enter sale price: "))
    Sale[item] = price
# Merge
Catalog.update(Sale)
print("Updated Catalog:", Catalog)
```

```
Enter number of catalog items: 3
Enter item name: headphones
Enter price: 5000
Enter item name: mouse
Enter price: 3000
Enter item name: controller
Enter price: 1500
Enter number of sale items: 1
Enter sale item name: keyboard
Enter sale price: 4000
Updated Catalog: {'headphones': 5000, 'mouse': 3000, 'controller': 1500, 'keyboard': 4000}
```

5# A Math Learning App needs a quick lookup table for squares to speed up calculations for students. Write a Python script to generate a dictionary, take user input for range of keys: • Keys: Integers from 1 to 15 (inclusive). • Values: The square of the key (key2). Example: • Output Snippet: {1: 1, 2: 4, 3: 9 ... 15: 225}

```
n = int(input("Enter the range: "))
squares = {}
for i in range(1, n + 1):
    squares[i] = i * i
print("Squares Dictionary:")
print(squares)
```

```
Enter the range: 10
Squares Dictionary:
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

6# Implement the Matrix Chain Multiplication problem using dynamic programming in Python.

```
def matrix_chain(p):
    n = len(p) - 1
    dp = [[0]*(n+1) for _ in range(n+1)]
```

```
    for length in range(2, n+1):        # chain length
        for i in range(1, n-length+2):
            j = i + length - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]
                dp[i][j] = min(dp[i][j], cost)

    return dp[1][n]

# input
n = int(input("Number of matrices: "))
p = list(map(int, input("Enter dimensions (n+1 numbers): ").split()))

print("Minimum multiplications:", matrix_chain(p))
```

```
Number of matrices: 3
Enter dimensions (n+1 numbers): 10 20 30 40
Minimum multiplications: 18000
```

7# A GPS Navigation System needs to calculate the fastest route between cities connected by highways with different distances (weights). Write a Python program to find the Shortest Path from a starting node to a destination node in a weighted graph using Dijkstra's Algorithm.

```
graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'A': 4, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 8},
    'D': {'B': 5, 'C': 8}
}
start = input("Enter start node: ")
```

```
dist = {}
visited = {}
for node in graph:
    dist[node] = float('inf')
    visited[node] = False

dist[start] = 0
for _ in graph:

    min_node = None
    min_distance = float('inf')

    for node in graph:
        if not visited[node] and dist[node] < min_distance:
            min_distance = dist[node]
            min_node = node
    visited[min_node] = True
    for neighbor in graph[min_node]:
        new_distance = dist[min_node] + graph[min_node][neighbor]
        if new_distance < dist[neighbor]:
            dist[neighbor] = new_distance

print("Shortest distances:", dist)
```

```
Enter start node: A
Shortest distances: {'A': 0, 'B': 3, 'C': 2, 'D': 8}
```